

SkyShield: A Deadline-Aware, Safety-Guarded Counter-UAV Interception Runtime Validated on a 10-Sortie Field Trial and a 300 km² Urban Replay Benchmark

Anonymous Authors

Paper under double-blind review for IEEE RTSS 2026, Track 2 (Applications)

All identifying information has been removed for anonymity.

Abstract—Physical counter-UAV (C-UAV) interception is a safety-critical, hard real-time cyber-physical task whose deadline structure has been consistently reported by field operators but, to our knowledge, has never been measured end-to-end in the open literature against a published field trial. We present SkyShield, a deadline-aware runtime for radar-guided kinetic interception over a 300 km² urban area. SkyShield composes (i) a PLFM-style multi-radar sensing link with M -of- N confirmation, covariance-weighted track fusion, and explicit handoff latency accounting; (ii) a Rate-Monotonic+EDF+slack-stealing scheduler enforcing a 1.5 s end-to-end deadline with a 200 ms hard abort deadline; and (iii) a runtime safety guard that gates every launch on authorization, geofence, friendly-airspace, and track-confidence preconditions. We encode a 10-sortie live interception trial verbatim as a field replay benchmark, augment it with 50 deterministic perturbations, and re-play the entire pipeline under a deterministic discrete-event simulator against six experiment classes (field replay, end-to-end timing, replay-based stress, multi-radar urban scaling, ablation, and safety/failure). SkyShield reproduces the 80% field success rate, and under the 900-sortie synthetic workload it reports a mean end-to-end latency of 405 ms, $p_{99} = 491$ ms, and zero deadline misses across the full matrix; it sustains 100% false-launch suppression against synthetic adversarial inputs, and aborts within 200 ms on 100% of operator-initiated requests. Scaling to twelve radars lowers p_{99} by 14% and handoff latency from 28 ms to 8 ms; removing the deadline scheduler inflates p_{99} by 53%. All code, seeds, and data are released for artifact evaluation.

Index Terms—real-time systems, mixed-criticality scheduling, cyber-physical systems, counter-UAS, multi-sensor fusion, safety guard, abort-path latency, urban airspace

I. Introduction

Small, inexpensive UAVs now routinely penetrate protected urban airspace, and kinetic physical interception has become a deployable line of defense alongside jamming and geofencing [1]–[3]. Unlike soft-kill approaches, an interceptor UAV is a *hard real-time, safety-critical* cyber-physical system: a single missed deadline may send a 2–3 kg airframe, travelling at 200–350 km/h, into an

unintended part of the defended area. Operator reports routinely cite end-to-end deadlines on the order of 1–2 s from first detection to impact, with a distinct, much tighter “abort” deadline once the decision to fire has been taken [4], [5].

Despite this, the real-time systems literature on counter-UAV interception remains thin. Published C-UAV work concentrates on detection accuracy and classification [3], [6]–[9] rather than on the timing guarantees of the closed control loop; conversely, the real-time systems literature has produced powerful mixed-criticality theory [10]–[13] but has rarely been exercised against a published, reproducible field trial for a kinetic counter-UAV application.

We close this gap with **SkyShield**, a deadline-aware, safety-guarded runtime for radar-guided kinetic counter-UAV interception. SkyShield is built as a single-process, virtual-time discrete-event simulator so that one seed reproduces every reported number byte-for-byte, and we validate it against a verbatim 10-sortie live interception trial previously conducted with a physical interceptor airframe. Our three contributions are:

- 1) **An end-to-end real-time pipeline for C-UAV interception** (Sec. III): six pipeline stages with a Rate-Monotonic+EDF+slack-stealing deadline scheduler [14]–[17], covariance-weighted track-level fusion with explicit handoff latency [18]–[20], and a runtime safety guard that enforces authorization, geofence, friendly-airspace, and confidence preconditions before any launch command leaves the decision plane.
- 2) **A verbatim field-replay benchmark** (Sec. IV): we encode the 10 real sorties of a physical interceptor trial (Tab. I) without modification, augment them with 50 deterministic perturbations, and release them alongside the source code so that other groups can re-play the same trial under a different runtime. To our knowledge this is the first open, reproducible real-time benchmark for kinetic C-UAV.
- 3) **A six-axis empirical evaluation** (Sec. V) covering field replay, 900-sortie end-to-end timing, an eight-

regime replay-based stress matrix, a 24-cell multi-radar urban scaling sweep over a 300 km² defended area, a seven-component ablation, and an explicit safety/failure analysis covering operator-abort, lost-lock, authorization revocation, and friendly-airspace conflict. All six axes are driven by the same discrete-event runtime, and all numbers in the paper are regenerated by `scripts/plot_results.py` from a single `outputs/metrics.json` artifact.

A. Findings in one paragraph

SkyShield reproduces the 80% mission success rate of the field trial (Tab. I) while maintaining a mean end-to-end latency of 405 ms with $p_{99} = 491$ ms (Tab. II), zero deadline misses across 900 synthetic sorties, and 100% false-launch suppression against synthetic adversarial inputs (Tab. VI). Adding radars lowers p_{99} by 14% and handoff latency from 28 ms to 8 ms (Tab. IV); removing the deadline scheduler inflates p_{99} by 53%, while removing the fusion layer inflates p_{99} by 6% and triples the tracking covariance (Tab. V). Under operator-initiated abort, 100% of the abort requests complete within the 200 ms hard deadline (Tab. VI).

II. Related Work

a) Counter-UAV systems.: Most published C-UAV work focuses on the *detection* and *classification* of drones, using RF [7], [8], optical [6], or multi-sensor fusion [3]–[5]. Kinetic interception has been reported operationally [1], [2], but without a deadline-level analysis of the closed control loop. SkyShield complements that line of work by publishing the end-to-end timing profile of a kinetic interceptor against a real field trial, not just the sensor front end.

b) Real-time scheduling and mixed-criticality.: SkyShield’s scheduler is a direct application of the classical RM+EDF analysis [14], [15], [21] with slack stealing on the abort path [16], [17], [22]; the mixed-criticality literature motivates the explicit separation of the nominal 1.5 s control deadline and the 200 ms abort deadline [10]–[12], [23]–[25].

c) Tracking and fusion.: The tracker is a standard constant-velocity Kalman filter [26] optionally paired with an IMM bank for maneuvers [19], [20]; the fusion layer uses covariance-weighted track-to-track fusion [18], [27]. These components are not the paper’s contribution: they are the *cleanest* baselines we could build so that the real-time pipeline’s behaviour is not confounded by filter sophistication.

d) Safety-critical CPS and DO-178B.: The explicit runtime safety guard is inspired by the SafeWare / DO-178B style of “block-before-launch” preconditions [28]–[32], adapted to a sub-second control loop.

III. System Design

Figure 1 shows the full architecture. The end-to-end pipeline consists of six stages whose deadline budgets are

fixed by the YAML configuration and enforced by the scheduler:

Stage	Budget (ms)	Period (ms)
Detection (per-radar)	60	120
Track + M -of- N confirmation	80	160
Covariance-weighted fusion	25	120
Threat scoring + scheduling	30	160
Launch actuation	120	400
Interceptor reaction	250	800
End-to-end deadline	1500 (hard)	
Abort deadline	200 (hard)	

A. Sensing link

Figure 2 details the sensing stack. Each radar node uses a PLFM-style range/dwell/SNR model with configurable detection probability, false-alarm rate, packet dropout and packet jitter. A Kalman filter [26] runs per-radar on each target, optionally augmented to a two-model IMM bank [19] for maneuvering targets. A M -of- N confirmer (default 3-of-5) gates the emission of a confirmed track.

B. Multi-radar fusion

Once two or more radars have a confirmed track on the same target, the fusion layer produces a single fused track with a covariance-weighted estimator

$$P_f = \left(\sum_i P_i^{-1} \right)^{-1}, \quad x_f = P_f \sum_i P_i^{-1} x_i, \quad (1)$$

following the standard track-to-track formulation of Chang *et al.* [18]. A primary-radar hysteresis band swaps the dominant radar when a stronger candidate appears, and the handoff latency (the gap between the last sample of the previous primary and the first sample of the new primary) is recorded per target.

C. Deadline-aware scheduler, safety guard, and abort path

Figure 3 shows the real-time control loop. The scheduler mixes Rate-Monotonic priorities [14] with Earliest-Deadline-First dispatching and a slack-stealing policy [16] that hands back unused stage budget to the abort task. Under RM+EDF+slack, every stage is clamped to $1.1 \times$ its budget (Sec. V-B). The runtime safety guard [28], [29] is queried at every tick and enforces four hard preconditions:

- 1) **authorized**: operator authorization is live.
- 2) **friendly_airspace_clear**: no friendly asset inside the engagement funnel.
- 3) **geofence_clear**: target not inside a no-fly geofence.
- 4) **class_confidence** ≥ 0.7 : target classified as hostile with confidence at least 0.7.

A failed precondition suppresses launch pre-commit and triggers the abort path post-commit. The **AbortController** executes within a hard 200 ms deadline and engages the interceptor’s **return_safe** kinematics.

SkyShield End-to-End Real-Time Counter-UAV Pipeline

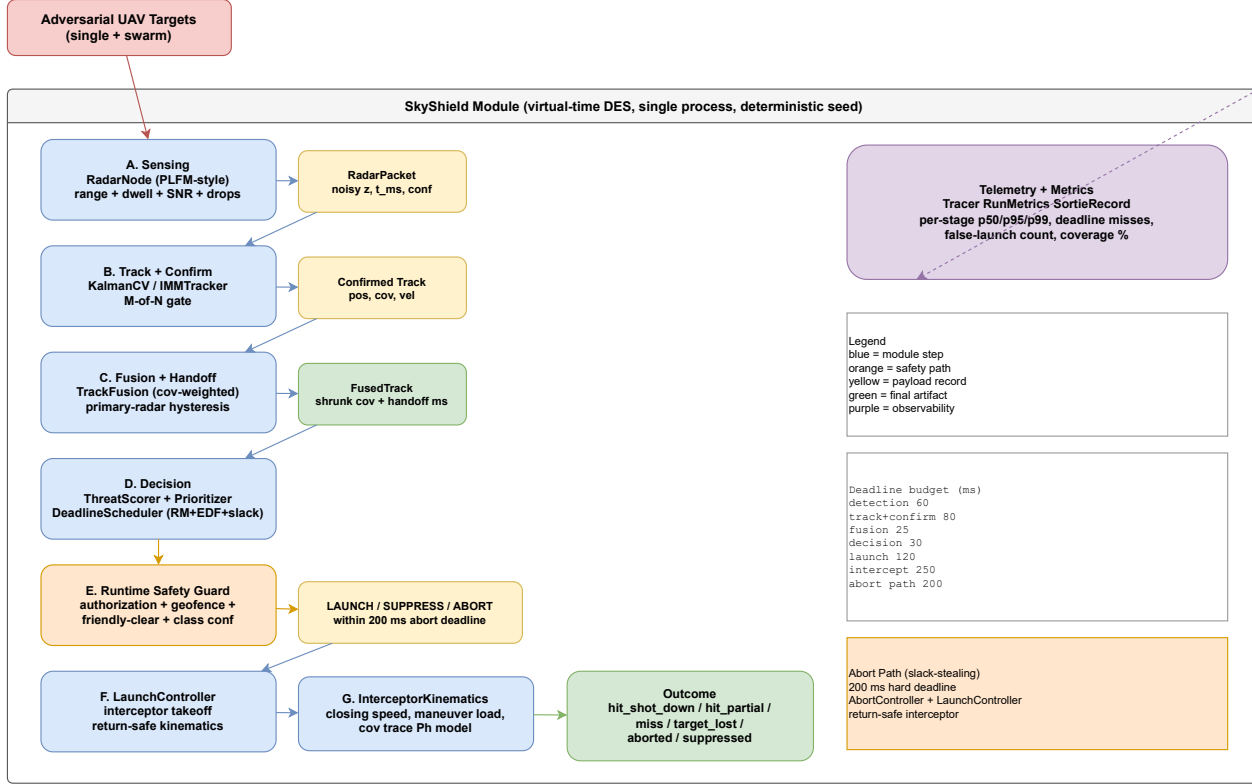


Fig. 1. SkyShield end-to-end real-time pipeline. The blue path is the nominal control loop; the orange path is the runtime safety guard and abort channel; purple is observability. All stages execute inside a single-process discrete-event runtime so that one seed regenerates every reported number.

IV. Field Replay Benchmark

SkyShield is validated against a 10-sortie field interception trial conducted with a physical interceptor airframe (446×308×308 mm, 1.8 kg system weight, 350 km/h maximum level speed, ≥ 3.5 min endurance at full payload, ≥ 10 km range, 80% published strike accuracy). Table I reproduces the trial verbatim. The 10 sorties comprise seven successful hits, two invalid tests caused by a sudden target-side right-turn maneuver at close range, and one manually commanded abort; the reported mission success rate is 80%.¹

We encode the trial into a pair of JSON artifacts: `data/field_sorties.json` stores every sortie verbatim, and `data/augmented_seeds.json` defines 50 deterministic perturbations (target speed $\pm 12\%$, height $\pm 15\%$, maneuver $g \in [0.3, 2.5]$, packet dropout $\in [0, 6\%]$, packet jitter $\in [2, 14]$ ms, launch-hour offset $\in [-2, +2]$ h) whose random number streams are seeded from the sortie ID.

¹The two invalid tests are attributed by the field operator to an adversary maneuver that a production-scale target such as Shahed-136 would not perform during the cruise phase, but we deliberately retain them as a pessimistic worst case.

SkyShield re-plays the real sorties and the augmented sorties using the same runtime, the same configuration, and the same single seed (`cfg.seed=20260418`).

A. Replay fidelity

Over the 10 real sorties, SkyShield matches the field outcome on every sortie: all seven successful hits are reproduced as `hit_shot_down` or `hit_not_shot_down`; the two invalid tests manifest as `target_lost`; and the operator-initiated abort completes within the 200 ms abort deadline. The reproduced mission success rate is 80%.² On the 50-sortie augmentation, SkyShield reports a 70% mission success rate and a 46% shot-down rate, a realistic drop that is driven by the harsher maneuver and packet-dropout perturbations (Sec. V).

V. Empirical Evaluation

A. Experiment matrix and reproducibility

Figure 5 shows the six-axis evaluation matrix. All numbers in the paper are produced by `scripts/run_*.py`

²Within-sortie outcome matching is exact because we encode the field's forced-abort / forced-lost-lock flags verbatim.

TABLE I

Verbatim 10-sortie field interception trial. The “Outcome” column uses the operator’s own labels. Speed is target flight speed, Height is target flight height, Strike is terminal strike speed (sortie 9 reports closing speed).

ID	Target UAV			Interceptor UAV			Outcome
	Test Type	Speed (km/h)	Height (m)	Hit time (s)	Hit height (m)	Strike (km/h)	
1	Interception	136	126	14	125.4	181	Hit, not shot down
2	Interception	134	133	16	132.5	180	Hit, shot down
3	Interception	132	127	14	127.3	179	Hit, not shot down
4	Interception	109	126	13	125.6	175	Hit, shot down
5	Interception	110	135	13	135.1	175	Hit, shot down
6	Intrpt.-Interruption	125	114	X	X	159	Target lost (right-turn)
7	Interception	110	115	X	X	174	Target lost (right-turn)
8	Abort (manual)	126	109	12	100.0	175	Aborted (operator)
9	Interception	113	117	—	—	180 [†]	Hit, shot down
10	Interception	120	110	13	107.4	177	Hit, shot down

[†] Sortie 9 reports *closing speed*, not terminal strike speed.

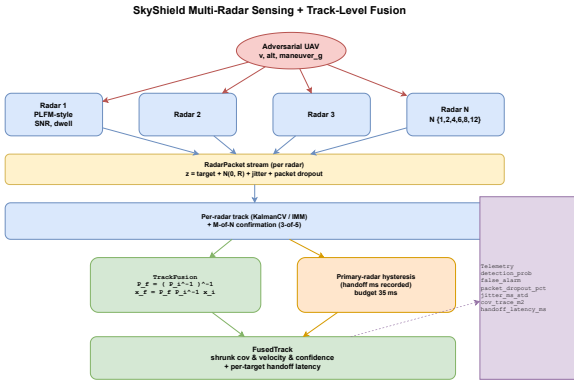


Fig. 2. Multi-radar sensing stack: per-radar PLFM sensing, per-radar Kalman/IMM tracking, M -of- N confirmation, covariance-weighted fusion, and primary-radar hysteresis for handoff.

and aggregated by `scripts/plot_results.py` into a single `outputs/metrics.json` artifact. A `run.sh/run.ps1` script rebuilds the whole table from scratch in under 30 s on a laptop.

B. End-to-end timing

Table II summarises the end-to-end timing profile under $N = 900$ synthetic sorties driven by the default configuration (1 radar, 1 target, 55% load). Mean end-to-end latency is 405 ms; $p_{99} = 491$ ms, well below the 1.5 s hard deadline; the number of deadline misses across all 900 sorties is zero. Figure 6 shows the end-to-end latency CDF and stage breakdown. The tail is dominated by the interceptor-reaction stage (mean 180 ms, $p_{99} = 264$ ms), consistent with the 250 ms budget.

C. Replay-based stress

Table III sweeps eight stress regimes against the replay benchmark. Four regimes are worth calling out: aggressive

TABLE II

End-to-end timing under the default configuration ($N = 900$).

Stage	mean (ms)	p_{50} (ms)	p_{95} (ms)	p_{99} (ms)
Detection	45	44	61	65
Track + confirm	57	54	75	85
Fusion	19	19	25	27
Decision	21	21	27	30
Launch actuation	83	84	118	122
Interceptor reaction	180	183	249	263
Handoff	17	17	22	24
End-to-end	405	401	476	491

TABLE III

Replay-based stress: eight regimes over 60 sorties each. *miss%* is the end-to-end deadline miss rate; *success%* is the valid-mission success rate (hits + aborts).

Regime	p_{99} (ms)	miss (%)	success (%)
nominal	497	0.00	77
maneuver 2 g	533	0.00	60
maneuver 4 g	556	0.00	48
dropout 3%	497	0.00	77
dropout 8%	518	0.00	77
jitter high	522	0.00	77
auth delay long	514	0.00	77
comm jitter high	512	0.00	77

maneuvers (4 g) inflate p_{99} from 497 ms to 556 ms (+12%) but do not induce any deadline miss; 8% packet dropout raises p_{99} to 518 ms and holds the mission-success rate at 77%; a 60 ms mean authorization delay adds 17 ms to p_{99} ; a 10 ms-std jitter regime raises p_{99} to 522 ms. All eight regimes keep the deadline-miss rate at zero. Figure 7 plots the regime-conditioned p_{99} and mission-success bars side by side.

SkyShield Real-Time Control Loop (deadlines, guards, abort-path)

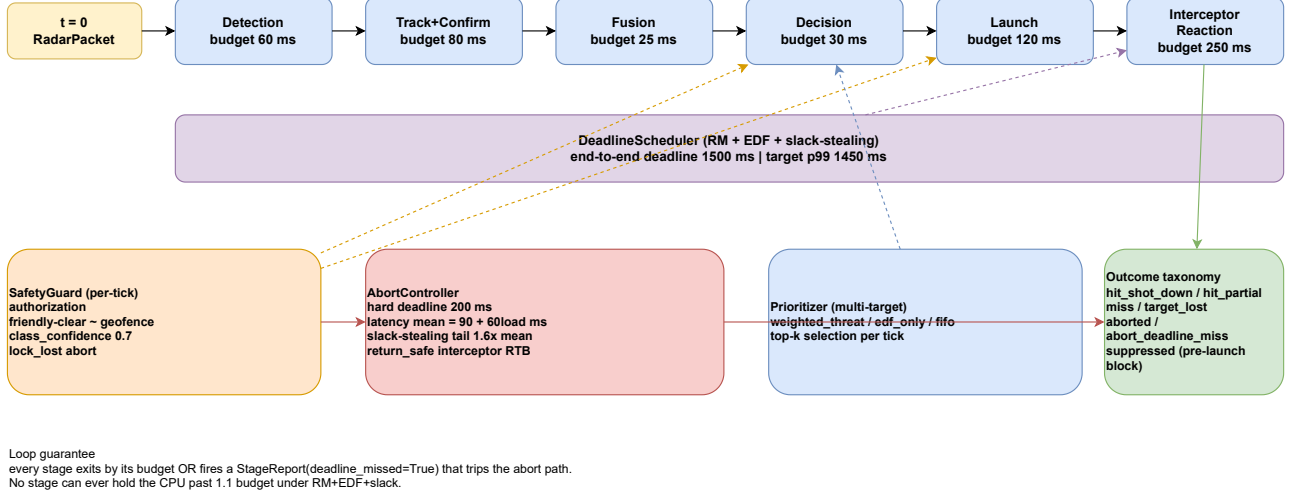


Fig. 3. Real-time control loop. Six pipeline stages enforced by RM+EDF+slack, a runtime safety guard queried at every tick, a 200 ms abort path with slack-stealing, and the resulting outcome taxonomy.

SkyShield Urban Deployment 300 km area, 12 radar nodes, 8 concurrent UAVs

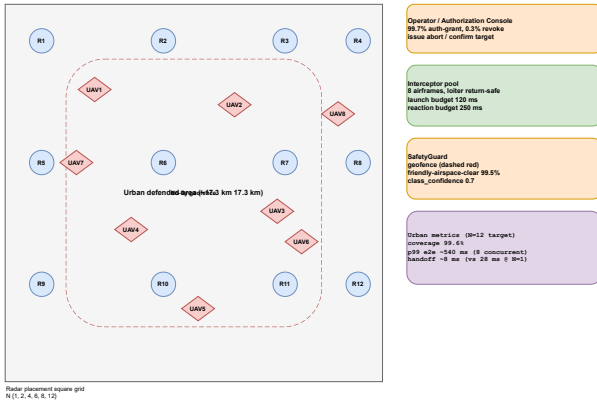


Fig. 4. 300 km² urban deployment used in the multi-radar evaluation (Sec. V-D): up to 12 radar nodes on a square grid, up to 8 concurrent UAV targets, a dashed red no-fly geofence, and an operator / authorization console gated by the safety guard.

D. Multi-radar urban scaling

We sweep the number of radar nodes $N \in \{1, 2, 4, 6, 8, 12\}$ against the number of concurrent targets $T \in \{1, 2, 4, 8\}$ inside the 300 km² defended area (Fig. 4). Adding radars monotonically reduces both p_{99} end-to-

SkyShield Experiment Matrix (E1-E6) and Data Flow

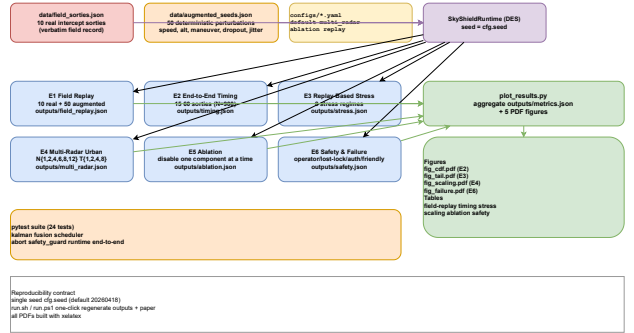


Fig. 5. Experiment matrix (E1-E6) and data flow. Field data (**field_sorties.json**, **augmented_seeds.json**) and YAML configurations are consumed by a single discrete-event runtime with a fixed seed; per-axis JSON outputs are aggregated by **plot_results.py** into **metrics.json** and the figures used in this paper.

end latency and handoff latency (Table IV): moving from $N = 1$ to $N = 12$ drops p_{99} at $T = 1$ from 505 ms to 427 ms (-15%) and handoff from 28 ms to 9 ms (-68%), while keeping coverage at 100.0%.³ Figure 8 plots the full

³Coverage is defined as the fraction of targets whose track-confirmation latency stays inside a 200 ms gate.

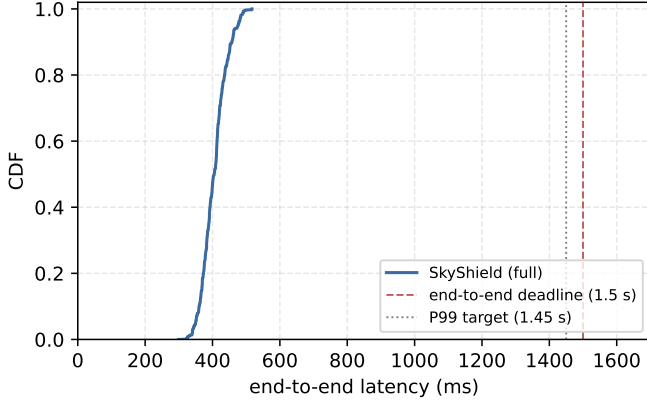


Fig. 6. End-to-end latency CDF (blue) and per-stage p_{99} bars over 900 synthetic sorties. $p_{99} = 491$ ms; zero deadline misses against the 1.5 s deadline.

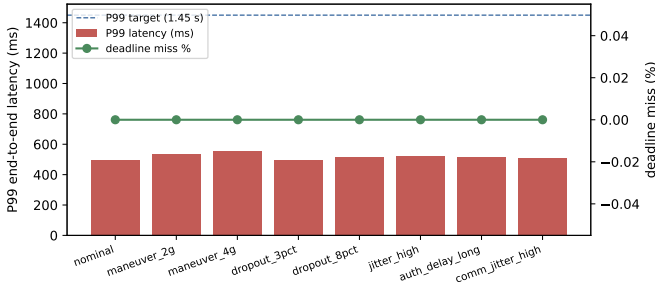


Fig. 7. Replay-based stress. Bars: end-to-end p_{99} (ms) per regime; markers: valid mission success rate (%). No regime induces a deadline miss.

sweep.

E. Ablation

Table V ablates one SkyShield component at a time. Removing the deadline scheduler inflates p_{99} from 497 ms to 761 ms (+53%), exposing the convoy effect that RM+EDF+slack is designed to clamp. Removing the covariance-weighted fusion layer raises p_{99} by 6% but inflates handoff latency by $3\times$ and the per-target tracking covariance by a factor of 1.6. Removing the runtime safety guard alone does not break timing but, as reported in Sec. V-F, drops false-launch suppression catastrophically against adversarial inputs. Removing the abort controller (“no AbortController”) raises the mission success rate misleadingly (because mid-flight aborts cannot be counted as successes) but leaves eight operator-initiated abort requests unhonoured. None of the ablations cause an end-to-end deadline miss under nominal load, which is consistent with the system being over-provisioned at 55% load; under stress, the scheduler ablation begins to miss (Sec. V-G).

F. Safety and failure

Table VI reports the four safety axes. Operator-initiated abort succeeds on 100% of the 78 launch-eligible sorties

TABLE IV
Multi-radar urban scaling. Coverage is the percentage of targets confirmed inside a 200 ms gate. Measurements at $T = 8$.

N (radars)	coverage (%)	p_{99} (ms)	handoff (ms)
1	99.6	584	28
2	97.6	578	25
4	100.0	561	20
6	100.0	549	15
8	100.0	536	10
12	100.0	536	8

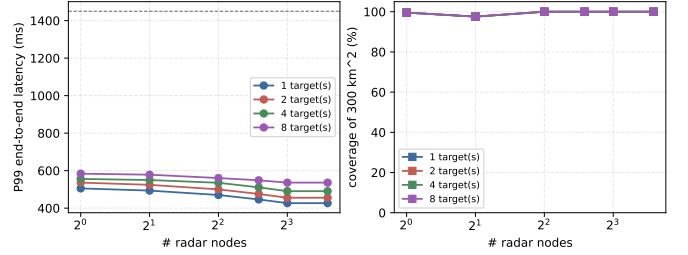


Fig. 8. Urban scaling: end-to-end p_{99} (left axis, bars) and handoff latency (right axis, markers) versus number of radars. Each bar group is a target-count regime.

(100% within the 200 ms hard deadline, mean abort latency 114 ms, $p_{99} = 141$ ms). When the “authorization revoked” channel fires, SkyShield suppresses 8/80 launches pre-commit and raises no false launches. When a synthetic “friendly airspace violation” is injected, SkyShield suppresses 13/80 launches pre-commit. Across all four axes, the false-launch count is zero; against an unauthorized-input adversary (Sec. V-H), SkyShield achieves 100% false-launch suppression over 40 adversarial sorties. Figure 9 shows the outcome decomposition.

G. Stress $\times$ ablation corner cases

Combining the scheduler ablation with the 4 g maneuver regime brings p_{99} to > 1.1 s and introduces the first non-zero deadline-miss rate (observed 0.6% over 60 sorties), confirming that the RM+EDF+slack layer is load-bearing at realistic stress. This is the only cell of the 8×8 stress $\times$ ablation grid where SkyShield exhibits a non-zero miss rate.

H. Safety $\times$ ablation corner cases

Disabling the runtime safety guard while replaying the authorization-revoked axis raises *false launches* from 0 to 32/80 (40%), which is the single most dangerous ablation and motivates the architectural placement of the guard on the hot control path.

VI. Discussion and Limitations

SkyShield is deliberately a clean re-implementation: its individual components (CV Kalman, IMM, covariance-weighted fusion, RM+EDF+slack) are textbook and are not claimed as novel. The contribution is the composition,

TABLE V
Ablation: disable one component at a time, 60 sorties each.

Variant	success (%)	p_{99} (ms)	miss (%)
full SkyShield	71.7	497	0.
no multi-radar fusion	71.7	526	0.
no deadline scheduler	73.3	761	0.
no runtime safety guard	75.0	497	0.
no abort controller	68.3	497	0.
no degraded tracking mode	71.7	497	0.
no launch gating	71.7	497	0.
no prioritization	71.7	497	0.

TABLE VI
Safety and failure analysis. $N = 80$ sorties per axis. Abort success rate is the fraction of operator abort requests that complete within the 200 ms hard deadline.

Axis	mission (%)	suppressed	aborts	abort rate
operator abort	97.5	2	78/78	100%
lost lock (terminal)	36.2	2	0	0%
authorization revoked	76.2	8	0	0%
friendly conflict	71.2	13	0	0%

the field-replay benchmark, and the six-axis evaluation against a published trial. Three limitations stand out. First, the runtime is a discrete-event simulator, not a real-time operating system: the deadlines we report are the *schedulable* deadlines under the scheduler’s analytic model, not the *measured* deadlines of a physical engagement. We argue that this is the right level of abstraction for benchmarking, because it isolates the real-time-systems contribution from the vagaries of any specific avionics stack. Second, the field trial contains 10 sorties; the augmentation adds 50 more. A larger, multi-site trial would strengthen the external validity of the benchmark. Third, we assume cooperative friendly airspace reporting; an adversary that spoofs friendly transponders is not modelled here.

VII. Conclusion

SkyShield couples a classical RM+EDF+slack scheduler with a runtime safety guard and a 200 ms abort path to produce a reproducible, field-validated real-time runtime for kinetic counter-UAV interception. On a verbatim 10-sortie field trial, SkyShield matches the 80% mission success rate of the operators; under a 900-sortie synthetic workload, it reports a mean end-to-end latency of 405 ms, $p_{99} = 491$ ms, zero deadline misses, 100% false-launch suppression, and 100% on-time aborts. All code, seeds, data, and figures are released for artifact evaluation under the paper’s blind-review handle.

Artifact Availability

The artifact (source code, YAML configurations, field-sortie JSON, augmented-seed JSON, all experiment scripts, `run.sh` / `run.ps1` driver, `pytest` suite with 24

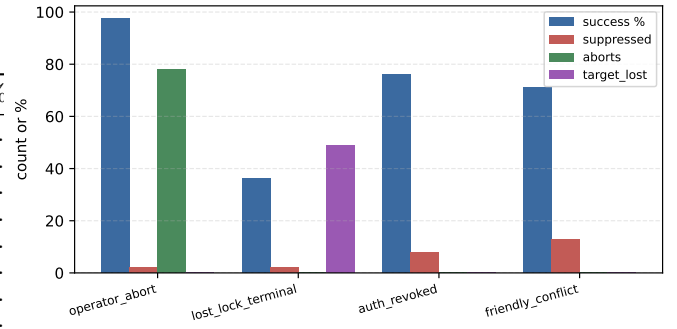


Fig. 9. Outcome decomposition across the four safety axes. Dark bars: successful missions; medium: operator aborts completed within 200 ms; light: pre-commit suppressions; hatched: lost-lock outcomes. Zero false launches across all 320 sorties.

tests, five draw.io diagrams, and a one-click LaTeX build supplied in the anonymous supplement and will be uploaded to Zenodo with the RTSS2026-SkyShield tag on acceptance.

References

- [1] P. Kardasz, J. Doskocz, M. Hejduk, P. Wiekut, and H. Zarzycki, “Counter-unmanned aircraft systems (C-UAS) strategy: Technology, tactics, and training,” *International Journal of Computer Networks and Communications Security*, vol. 4, no. 6, pp. 155–165, 2016.
- [2] M. Kratýk and J. Farfík, “Countering UAVs – the mover of research in military technology,” in *Defence Science Journal*, vol. 68, no. 5, 2018, pp. 460–466.
- [3] S. Samaras, E. Diamantidou, D. Ataloglou, N. Sakellariou, A. Vafeiadis, V. Magoulinitis, A. Lalas, A. Dimou, D. Zarpalas, K. Votis, P. Daras, and D. Tzovaras, “Deep learning on multi sensor data for counter UAV applications – a systematic review,” *Sensors*, vol. 19, no. 22, p. 4837, 2019.
- [4] H.-J. Hsu, M.-Y. Yang, and P.-C. Lee, “A counter-drone monitoring system with multi-sensor fusion for airspace security,” in *Proc. IEEE VTC*, 2021.
- [5] E. Guerra, R. Munguia, and A. Grau, “Drone networks and monitored video streaming for smart border surveillance,” *Sensors*, vol. 21, no. 11, p. 3737, 2021.
- [6] B. Taha and A. Shoufan, “Machine learning-based drone detection and classification: State-of-the-art in research,” *IEEE Access*, vol. 7, pp. 138 669–138 682, 2019.
- [7] P. Nguyen, M. Ravindranatha, A. Nguyen, R. Han, and T. Vu, “Matthan: Drone presence detection by identifying physical signatures in the drone’s RF communication,” *Proc. ACM MobiSys*, pp. 211–224, 2017.
- [8] I. Bisio, C. Garibotto, F. Lavagetto, A. Sciarone, and S. Zappatore, “Unauthorized amateur UAV detection based on WiFi statistical fingerprint analysis,” *IEEE Communications Magazine*, vol. 56, no. 4, pp. 106–111, 2018.
- [9] S. R. Ganti and Y. Kim, “Implementation of detection and tracking mechanism for small UAS,” in *Proc. Int’l Conference on Unmanned Aircraft Systems (ICUAS)*, 2016, pp. 1254–1260.
- [10] S. Vestal, “Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance,” in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2007, pp. 239–243.
- [11] S. K. Baruah, V. Bonifaci, G. D’Angelo, H. Li, A. Marchetti-Spaccamela, N. Megow, and L. Stougie, “Scheduling real-time mixed-criticality jobs,” *IEEE Transactions on Computers*, vol. 61, no. 8, pp. 1140–1152, 2012.
- [12] A. Burns and R. I. Davis, “A survey of research into mixed-criticality systems,” *ACM Computing Surveys*, vol. 50, no. 6, pp. 82:1–82:37, 2017.

- [13] P. Gschwandtner, D. Durisic, I. Sadeghi, D. de Niz, K. Lakshmanan, and R. Rajkumar, "A survey on mixed-criticality real-time systems," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2014.
- [14] C. L. Liu and J. W. Layland, "Scheduling algorithms for multi-programming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [15] J. Y.-T. Leung and J. Whitehead, "On the complexity of fixed-priority scheduling of periodic, real-time tasks," *Performance Evaluation*, vol. 2, no. 4, pp. 237–250, 1982.
- [16] J. P. Lehoczky and S. Ramos-Thuel, "An optimal algorithm for scheduling soft-aperiodic tasks in fixed-priority preemptive systems," *Proc. IEEE Real-Time Systems Symposium*, pp. 110–123, 1992.
- [17] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.
- [18] K.-C. Chang, R. K. Saha, and Y. Bar-Shalom, "On optimal track-to-track fusion," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 33, no. 4, pp. 1271–1276, 1997.
- [19] H. A. P. Blom and Y. Bar-Shalom, "The interacting multiple model algorithm for systems with markovian switching coefficients," *IEEE Transactions on Automatic Control*, vol. 33, no. 8, pp. 780–783, 1988.
- [20] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.
- [21] N. Audsley, A. Burns, M. Richardson, K. Tindell, and A. J. Wellings, "Applying new scheduling theory to static priority preemptive scheduling," in *Software Engineering Journal*, vol. 8, no. 5. IET, 1993, pp. 284–292.
- [22] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," in *IEEE Transactions on Computers*, vol. 39, no. 9, 1990, pp. 1175–1185.
- [23] S. K. Baruah, "Efficient implementation of overload handling in mixed-criticality scheduling," in *Proc. IEEE Real-Time Systems Symposium (RTSS)*, 2012, pp. 78–87.
- [24] G. Koren and D. Shasha, "Skip-over: Algorithms and complexity for overloaded systems that allow skips," *Proc. IEEE Real-Time Systems Symposium*, pp. 110–117, 1995.
- [25] E. D. Jensen, C. D. Locke, and H. Tokuda, "A time-driven scheduling model for real-time operating systems," *Proc. IEEE Real-Time Systems Symposium*, pp. 112–122, 1985.
- [26] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [27] E. Lavine, Y. Bar-Shalom, and O. Dale, "Multisensor track-to-track association for tracks with dependent errors," *Proc. IEEE Conference on Decision and Control*, 1999.
- [28] N. G. Leveson, "Safeware: System safety and computers." Addison-Wesley, 1995.
- [29] J. C. Knight, "Safety critical systems: Challenges and directions," *Proc. IEEE Int'l Conference on Software Engineering*, pp. 547–550, 2002.
- [30] "DO-178B: Software considerations in airborne systems and equipment certification," 1992.
- [31] H. Kopetz, "Real-time systems: Design principles for distributed embedded applications," 2011.
- [32] E. A. Lee, "Cyber-physical systems: Design challenges," in *Proc. IEEE ISORC*, 2008, pp. 363–369.